VRP Solver Comparison

Chihuahua Oxxo garbage-collection routing -- OR-Tools vs. genetic algorithm, 218 stops

218 stops served by both solvers under the same shift cap, capacity, and landfill-dump model.

Metric	OR-Tools	GA	Change
Total time	42.0h	43.1h	+2.7%
Total distance	1185.4 km	1236.0 km	+4.3%
Trucks used	6	6	+0.0%
Landfill trips	28	28	+0.0%

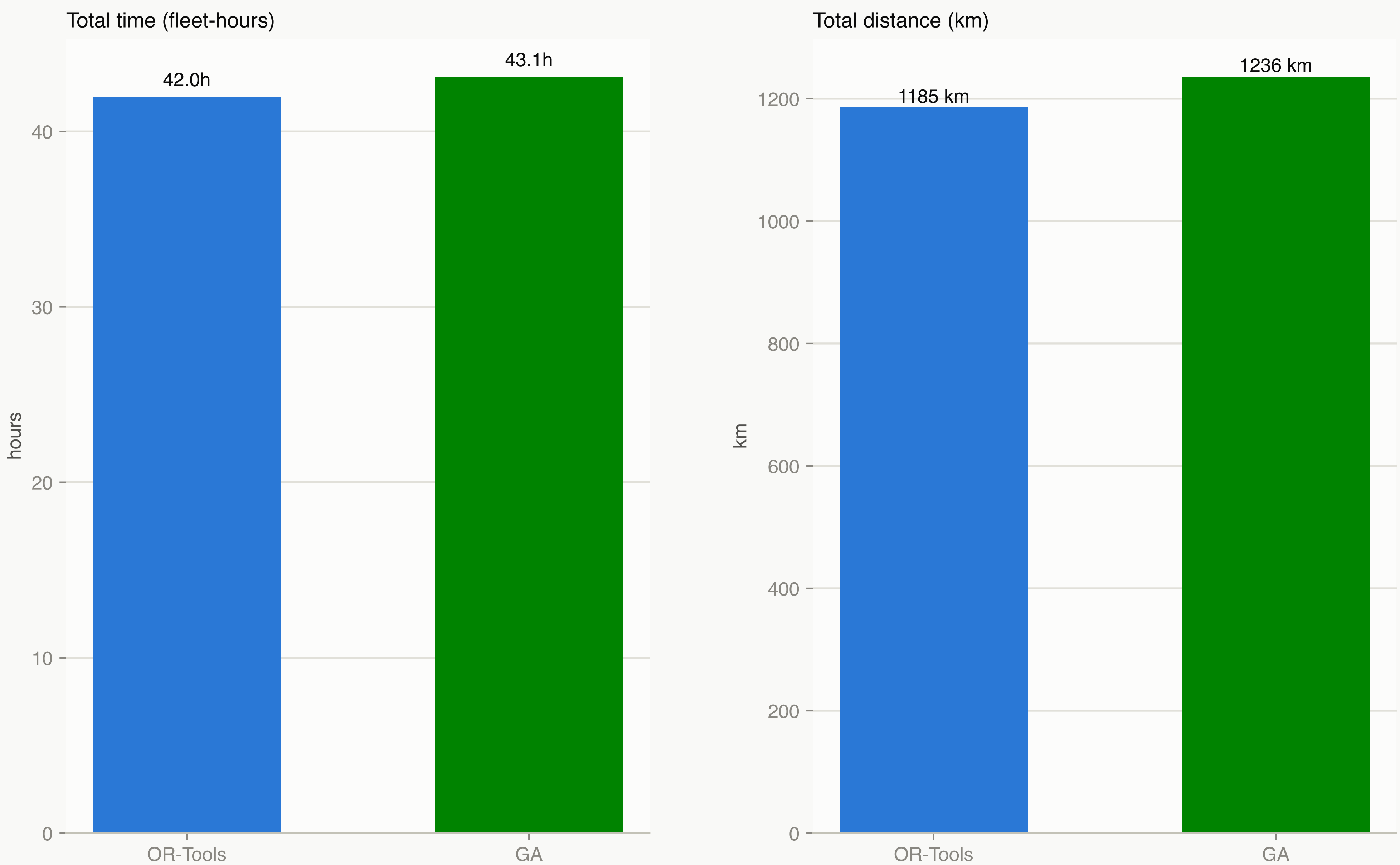
Read this report as:

OR-Tools remains the stronger baseline on this instance (fewer km, less total drive+service time), but the GA -- after adding nearest-neighbor + 2-opt construction and a tuned hyperparameter set (Code/tune_ga.py) -- closes most of the gap while matching OR-Tools on trucks used and landfill trips. Pages 2-4 compare the two solutions; page 5 shows the tuning sweep behind the GA's current defaults; pages 6-7 cover the pipeline/problem setup and open next steps.

Source: Data/routes-solution-summary.csv, Data/routes-solution-ga-summary.csv, Data/ga-tuning-trials.csv

Headline result

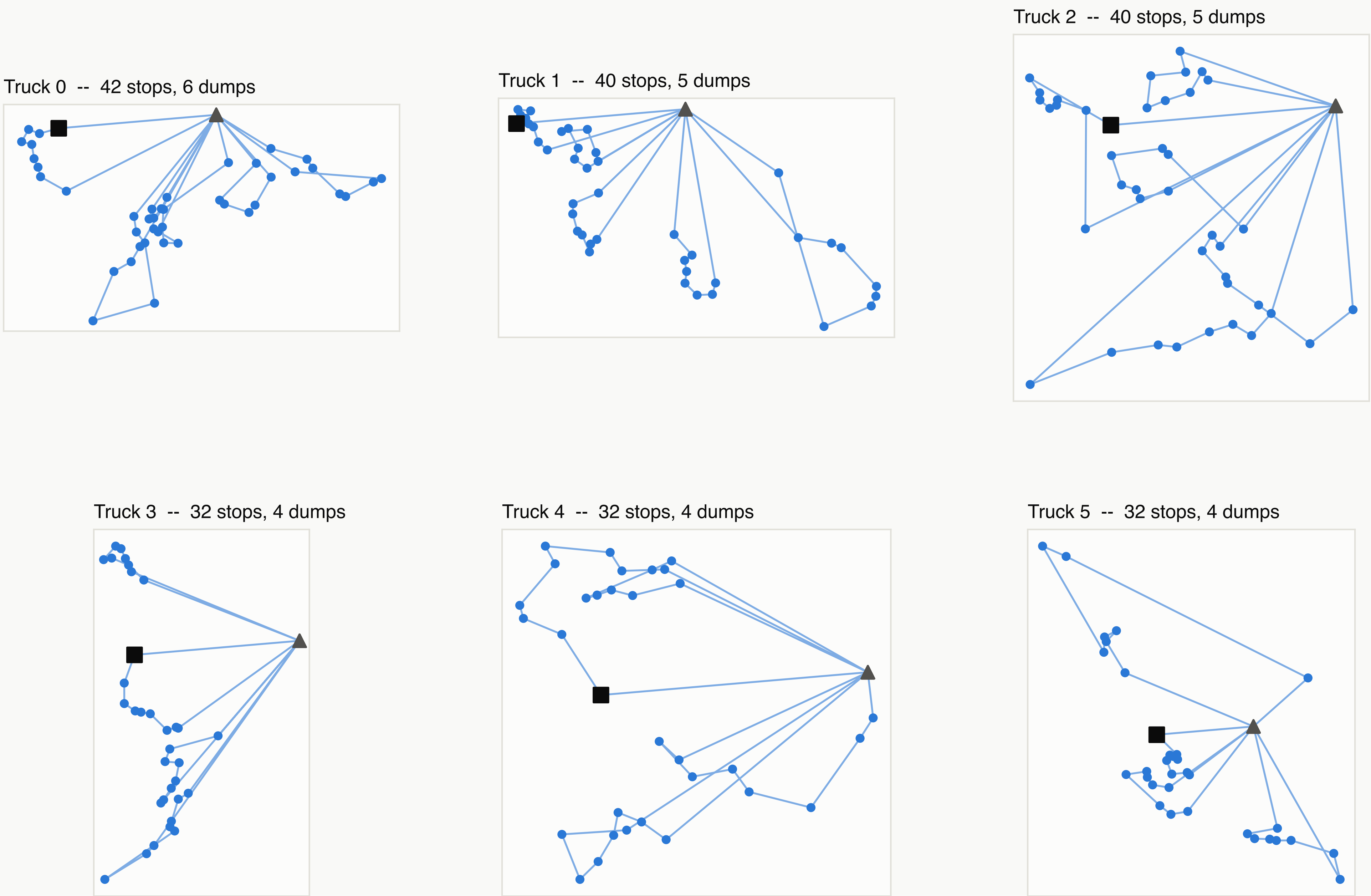
Same cost model, same instance, 60s solve budget each -- lower is better on both charts.



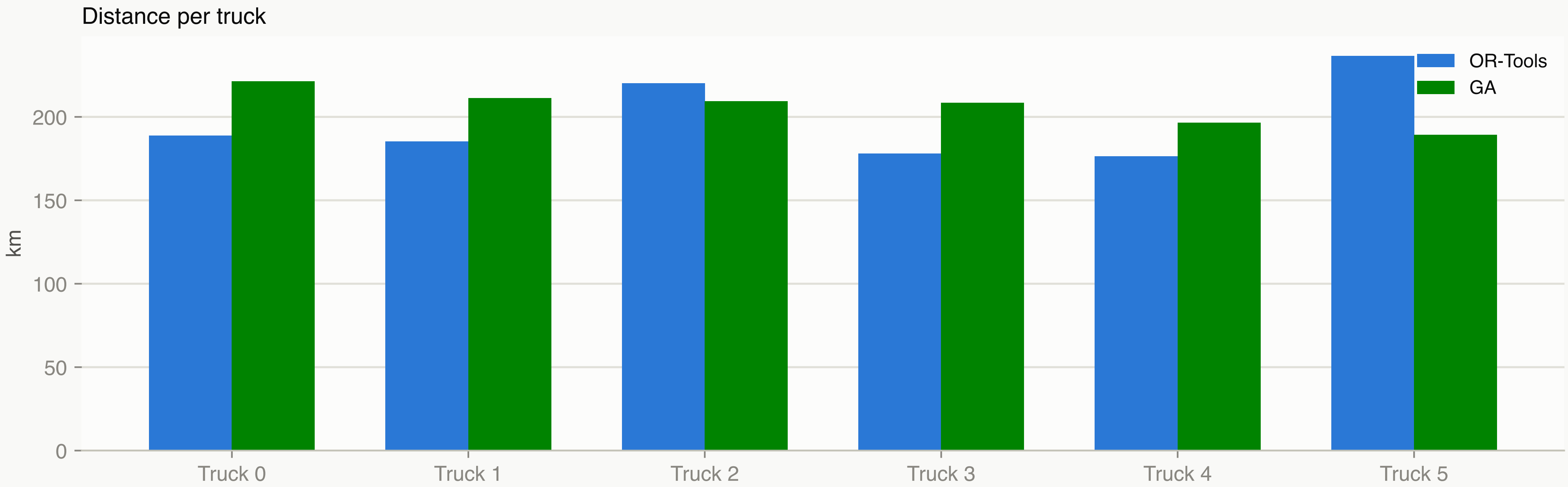
Both solvers use 6 trucks and 28 landfill trips for this instance -- fleet size and dump count are identical.

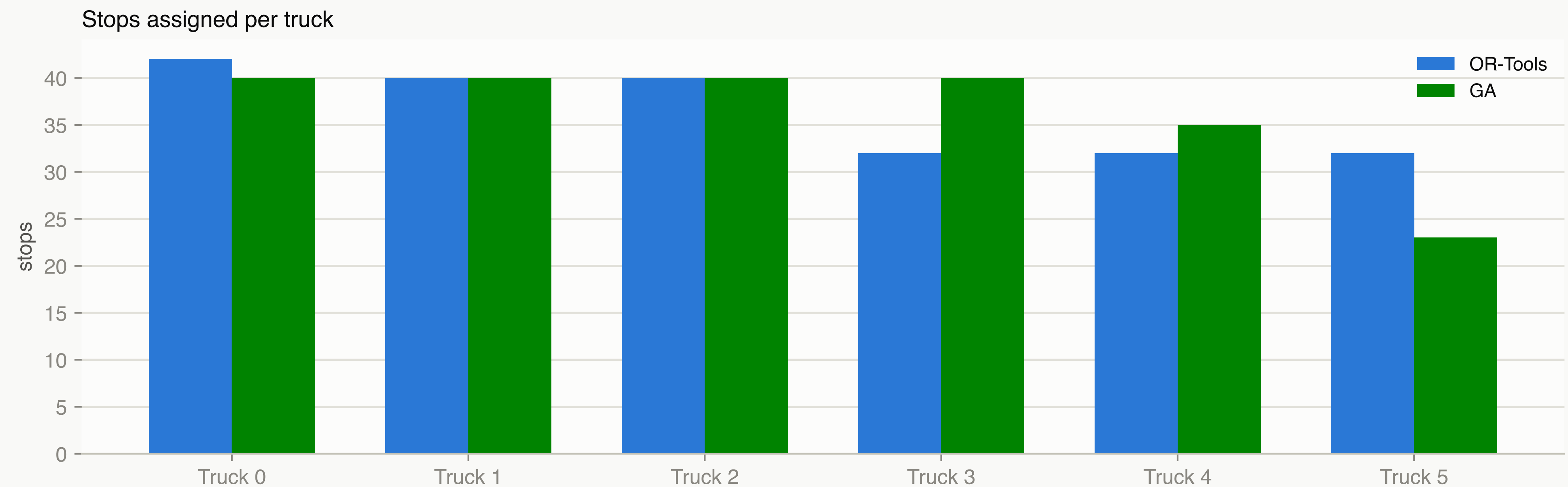
Route snapshot -- OR-Tools solution

Stop sequence per truck, straight-line (schematic, not street geometry). Depot = square, landfill dump = triangle, stop = dot. Full turn-by-turn detail: Data/mapa_rutas.html.



Per-truck breakdown

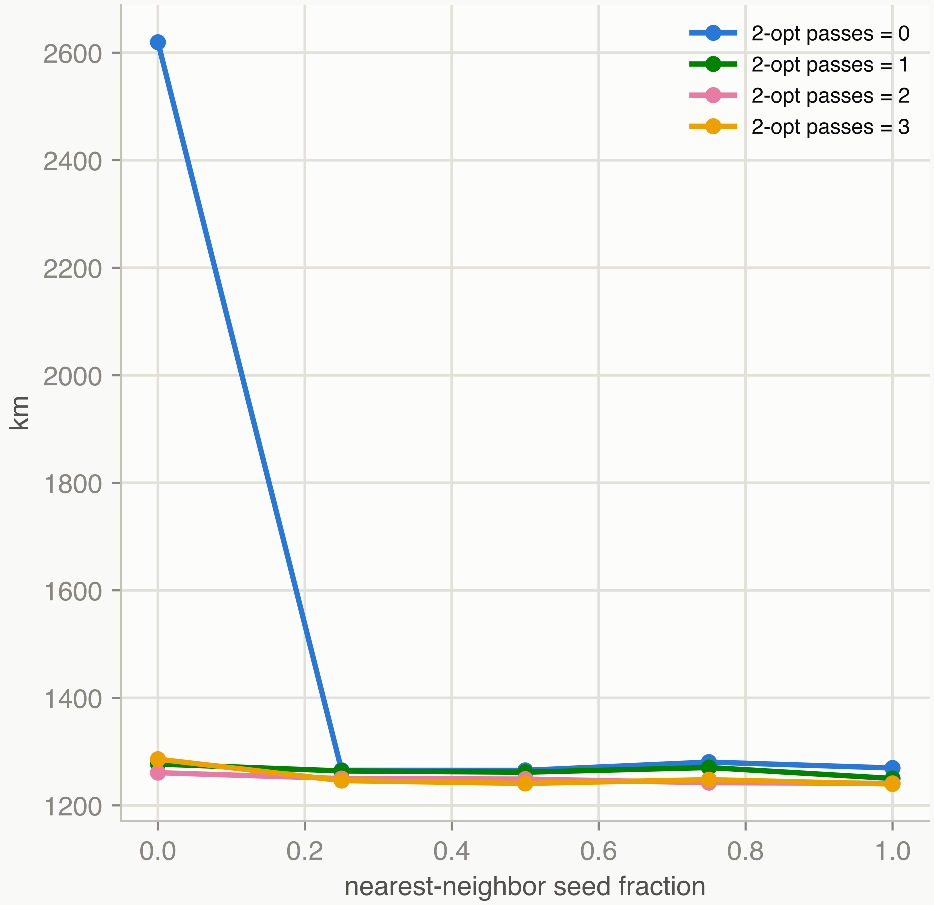




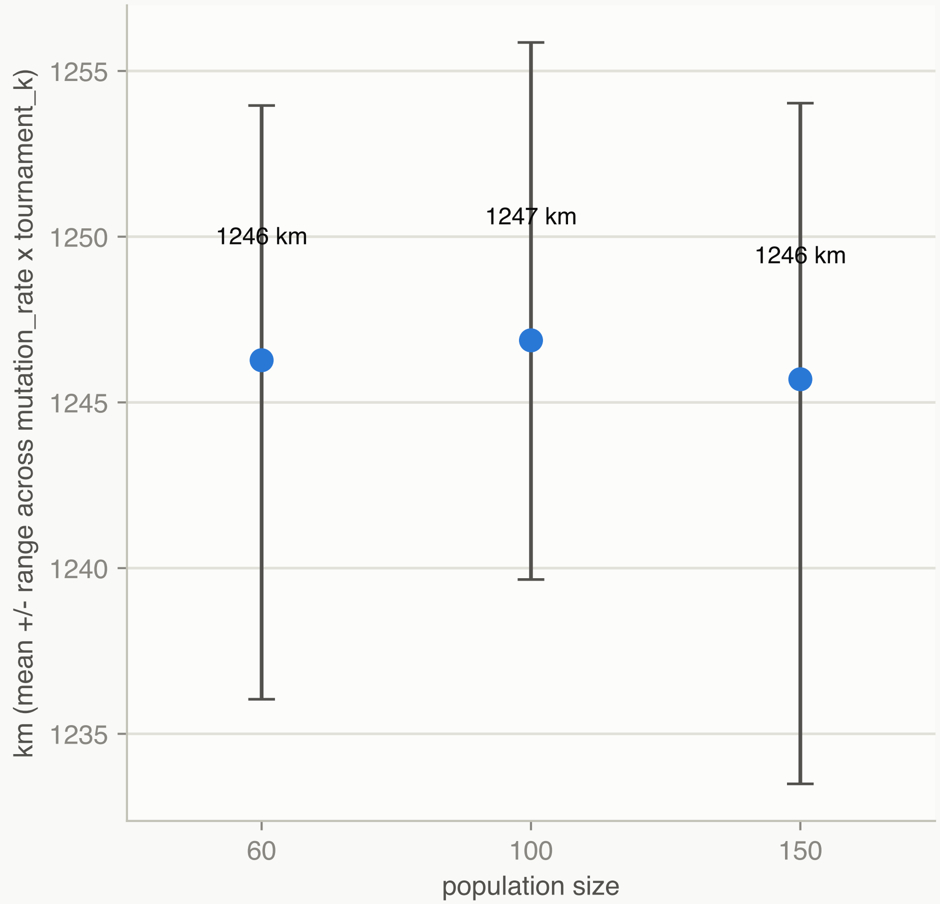
GA hyperparameter tuning

Sequential grid search, 47 trials at a 60s budget each, real 218-stop instance (Code/tune_ga.py).

Stage 1: construction beats loop tuning



Stage 2: loop knobs barely move the result



Winning configuration (minimizes total time; now solve()'s default):

`population_size=60`   `mutation_rate=0.15`   `tournament_k=5`   `nn_seed_fraction=1.0`   `two_opt_max_passes=3`

-> 43.12h total, 1236.0 km, 6 trucks, 28 landfill trips

Across the full grid, results span only ~43h07m-43h46m -- the 2-opt-refined construction step dominates solution quality far more than crossover/mutation/selection at this problem's scale.

Methodology & problem setup

Multi-trip vehicle routing problem (VRP) with intermediate landfill disposal, 218 Oxxo convenience stores in Chihuahua, Chihuahua, Mexico.

Problem constraints

Fleet size **10 trucks available**	Truck capacity **8 m3 per truck**	Shift cap **8h per truck per day**
Stop demand **0.9 m3 collected per stop**	Stop service time **180s per stop**	Landfill dump time **15 min per dump**

A truck returns to the landfill to empty its load whenever the next stop would exceed capacity (vrp_common.insert_dumps), so dump count is a consequence of the route, not a solver input.

Data pipeline

1. Geocode

Store addresses -> lat/lon (Data/oxxo-stops.csv)

>

2. Build matrices

OSRM /table -> time + distance matrices (build_time_matrix.py)

>

3. Solve

OR-Tools baseline and GA (solve_routes.py, solve_ga.py)

>

4. Visualize

Interactive Folium map, real street geometry (visualize_routes.py)

Solvers & validation

Both solvers consume the same points index and time/distance matrices and share one cost definition (vrp_common.py: summarize_route, insert_dumps, report_solution), so they compete on identical terms. OR-Tools is Google's constraint-programming VRP solver, run as a fixed-budget baseline. The GA builds each chromosome as a permutation of stops, seeds part of its initial population with nearest-neighbor tours refined by 2-opt, and reapplies 2-opt to elites each generation -- its hyperparameters were chosen by a 47-trial grid search (Code/tune_ga.py, see the tuning page). Correctness is covered by 46 pytest tests across both solvers, the shared cost model, and the benchmark harness.

Source: Claude/project_status.md, Code/vrp_common.py, Code/tune_ga.py

Recommendations & next steps

Open items from the current pipeline (Claude/project_status.md) worth acting on next.

HIGH

Expose the GA's tuned hyperparameters on the CLI.

population_size, mutation_rate, tournament_k, nn_seed_fraction, two_opt_max_passes, and elite_size are solve()'s defaults but not CLI flags -- re-tuning or A/B-testing them today means editing source, which is also what caused the earlier CLI/default-drift bug.

MEDIUM

Bring Code/pipeline/visualize_routes.py to parity with the flat script.

The flat Code/visualize_routes.py overlays OR-Tools and GA on one map; the class-based pipeline/ copy is still OR-Tools-only. Either port the overlay or document the divergence so it isn't mistaken for a bug.

MEDIUM

Clean up or remove the uncleaned exploratory scripts.

Code/geodata.py, visualize.py, and maptest.py have no CLI or docstring and aren't part of the active pipeline -- they're easy to mistake for live code by a new contributor.

LOW

Strip the Test1 placeholder row from Data/pipeline/oxxo-stops.csv.

Left over from the duplicate-id bug fix; harmless for pipeline/ runs but should not be mistaken for a real store when that directory's output is reviewed.

WATCH

GA is a validated fallback, not yet a replacement for OR-Tools.

Tuned, it trails OR-Tools by +4.3% km and +2.7% time on this instance -- worth keeping as a backup path (e.g. if OR-Tools' solve budget or licensing becomes a constraint) rather than switching over by default.